

Linux for DevOps

Lesson 4: Shell Scripting Basics

Shell Scripting Basics – Study Notes

Key Concepts

1. **Shebang & Execution** – The first line (`#!/usr/bin/env bash`) tells the system which interpreter to use, and `chmod +x script.sh` makes a file executable.
2. **Variables & Parameter Expansion** – Variables store data (`name="World"`), and `${var}` or `$var` expands them. Use quotes to preserve whitespace.
3. **Control Structures** – `if/elif/else`, `case`, `for`, `while`, and `until` loops allow you to branch logic and repeat actions.
4. **Functions & Scope** – `my_func() { ... }` groups reusable code; local variables prevent name clashes.
5. **Input/Output & Redirection** – `read`, `echo`, `printf` for I/O; `>`, `>>`, `<`, `<<`, `|` for redirecting streams.

Summary

Shell scripting is a powerful way to automate repetitive tasks on Unix-like systems. By writing scripts, you can combine command-line utilities, control flow, and variables into a single executable file. A typical script starts with a shebang, declares variables, and then uses loops or conditionals to perform actions such as file manipulation, network checks, or system monitoring. Functions help keep scripts modular, while redirection and piping let you manage input and output efficiently. Mastering these basics opens the door to more advanced topics like error handling, debugging, and writing robust, production-ready scripts.

Important Points

- **Always quote variable expansions** (`"$var"`) to avoid word splitting and globbing.
- Use `set -euo pipefail` at the top of scripts for safer execution (exit on error, unset vars, and pipe failures).
- `printf` is preferred over `echo` for predictable formatting.
- `[[ ... ]]` is the modern test command; avoid `[ ... ]` for complex conditions.
- Keep scripts portable by using `/usr/bin/env` in the shebang instead of hard-coding `/bin/bash`.
- Comment generously (`#`) and follow a consistent style (e.g., `snake_case` for variables).

Examples

1. Backup Script

```
#!/usr/bin/env bash
```

```
set -euo pipefail
SRC_DIR="/var/www/html"
DEST_DIR="/backup/$(date +%Y%m%d_%H%M%S)"
mkdir -p "$DEST_DIR"
```

Tar and compress the source directory

```
tar -czf "$DEST_DIR/website.tar.gz" -C "$SRC_DIR" .
echo "Backup completed: $DEST_DIR/website.tar.gz"
---
```

Explanation:

- `set -euo pipefail` ensures the script stops on errors.
- `\$(date +%Y%m%d\_%H%M%S)` creates a timestamped folder.
- `tar -czf` creates a compressed archive in the destination.

2. Service Status Checker

```
```bash
#!/usr/bin/env bash
services=("nginx" "mysql" "redis")
for svc in "${services[@]}; do
 if systemctl is-active --quiet "$svc"; then
 echo "$svc is running."
 else
 echo "$svc is NOT running."
 fi
done

```

\*Explanation:\*

- An array holds service names.
- `systemctl is-active --quiet` returns success if the service is active.
- The loop reports status for each service.

---

## Quick Review

1. What is the purpose of the shebang (`#!/usr/bin/env bash`) at the top of a script?
2. How do you declare a local variable inside a Bash function?
3. Write a `for` loop that prints numbers 1 to 5.
4. Explain the difference between `>` and `>>` redirection operators.
5. Why is it recommended to quote variable expansions?

---

## Further Reading

• **\*\*Advanced Bash Scripting Guide\*\*** – deep dive into arrays, associative arrays, and process substitution.

- **Error Handling & Debugging** – `set -x`, `trap`, and exit status conventions.
- **POSIX Shell vs Bash** – portability considerations.
- **Command Substitution & Process Substitution** – using `$(...)` and `<(...)`.
- **Writing Bash Libraries** – reusable functions and sourcing scripts.

Happy scripting!

